



计算机网络

Socket编程

徐敬东 张建忠

xujd@nankai.edu.cn

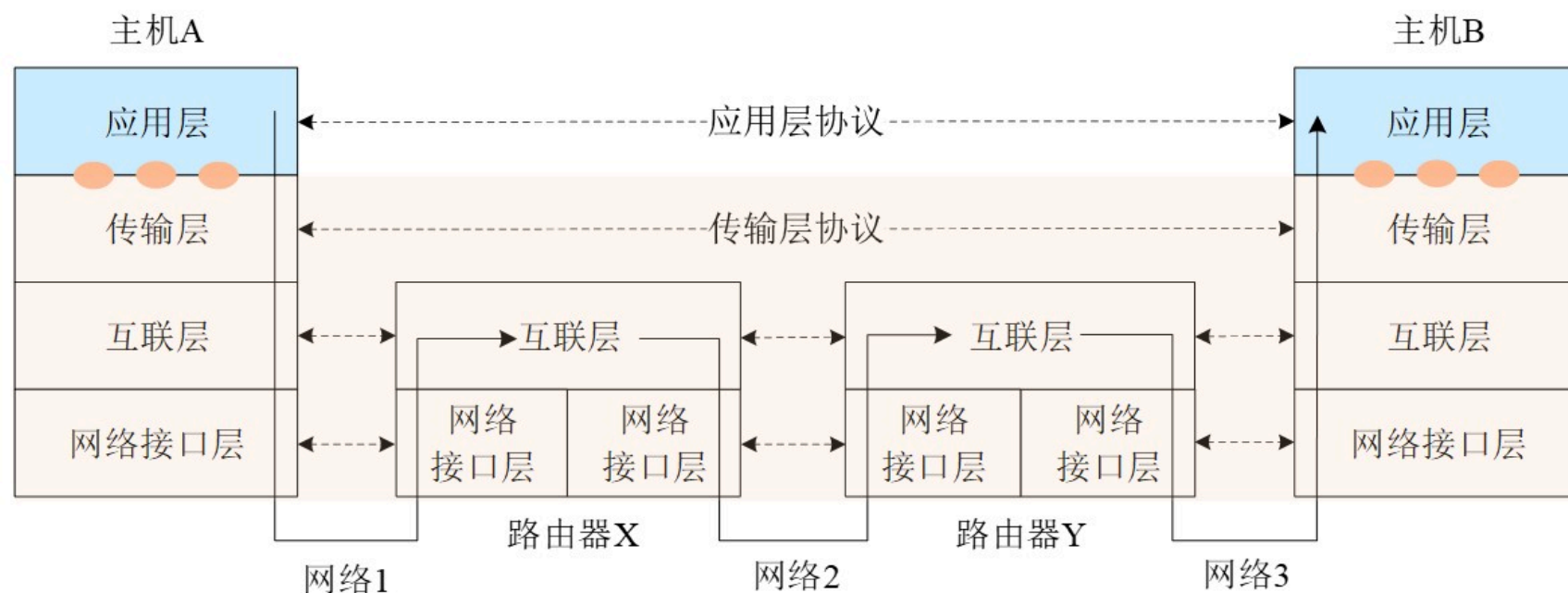
zhangjz@nankai.edu.cn

计算机网络与信息安全研究室

关于TCP/IP体系结构中传输层的描述中，正确的是

- ☐ A 包括MAC和LLC两个子层
- ☒ B 为应用进程之间提供端到端的传输服务
- ☐ C 屏蔽不同网络接口层的差异
- ☐ D 负责为IP数据包选择路径

- 传输层通过套接字为应用层提供端到端的传输服务
- 典型的传输层协议包括TCP和UDP
 - TCP通过流式套接字为应用层提供服务
 - UDP通过数据报套接字为应用层提供服务



■ 应用进程：实现应用层功能

- 在主机的用户空间实现
- 执行应用层协议，应用进程之间交换应用层协议数据单元（报文）

■ 应用进程交互模式

- 客户-服务器（Client-Server, C/S）模式
 - 客户和服务端为两个应用进程
 - 服务器进程处于守候状态，等待接收客户进程的请求并给出响应
 - 典型应用：DNS服务、WWW服务、电子邮件服务、文件传输服务等
- 对等（Peer-to-Peer, P2P）模式
 - 是一种分布式服务模式，最小化服务器的使用（或不使用）
 - 应用进程功能相同、地位相等，统称为对等方（peer）
 - 典型应用：即时通讯、网络电话等

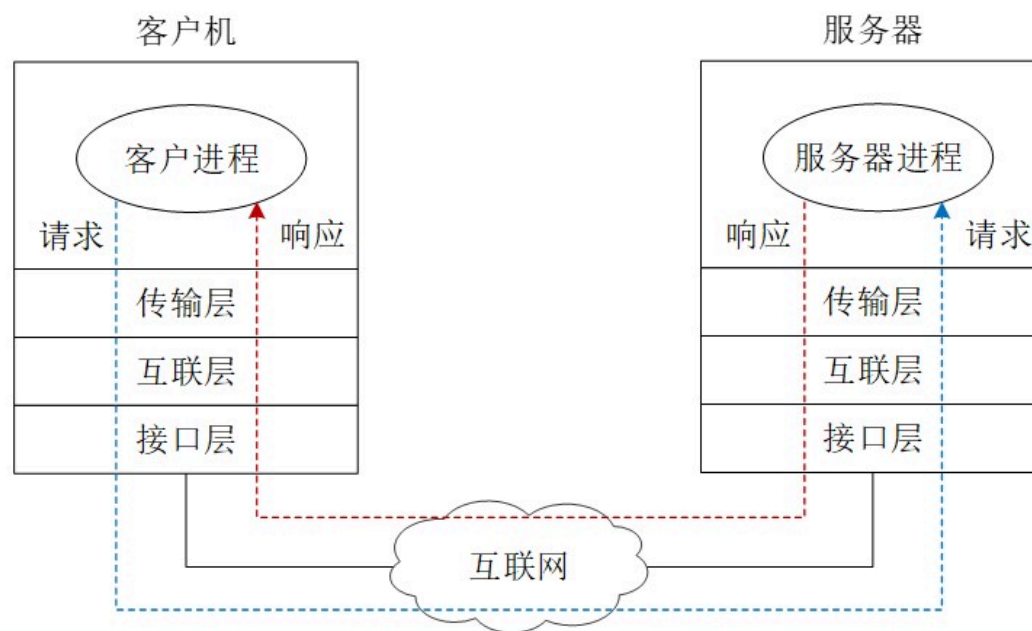
客户-服务器模式

■ 服务器进程

- 处于守候状态，等待客户进程的请求并给出响应
- 通常运行在TCP或UDP的公开端口上
- 运行服务器进程的主机一般具有固定的IP地址
- 客户进程通过IP地址和端口寻址服务器进程

■ 客户进程

- 可以随时运行，向服务器进程发送请求并接收响应
- 可以使用动态IP地址
- 客户进程之间不直接通信



应用进程标识方法

- **Q:** 在一台计算机中，进行用什么进行标识？
- 为了发送和接收消息，进程必须具有一个**标识符**
- 主机拥有一个唯一的32位的IPv4地址（或128位的IPv6地址）
- **Q:** 利用主机的IP地址表示主机中的进程是否可以？
 - **A:** 不可以。一个主机中可能同时运行着多个进程。

进程的地址标识（续）

- **进程标识符**：包括**IP地址**和**端口号**
- **端口号举例**：
 - Web服务器进程: 80
 - Email（SMTP）服务器进程: 25
- 为了向www.nankai.edu.cn的Web服务器发送消息，Web服务器需要使用的进程标识符为
 - IP地址：222.30.45.190（主机的IP地址）
 - 端口号：80

应用需要怎么的传输层服务？

■ 数据丢失率

- 音视频等应用可以容忍一定的数据丢失
- 文件传输、远程登录等应用要求100%的数据可靠

■ 时延

- 网络电话、交互游戏等应用对时延有一定的要求

■ 带宽

- 多媒体等应用需要一定的带宽保证
- 有些应用则是弹性的

常用应用对传输层的要求

应用	数据丢失	带宽	时延
文件传输	否	弹性	否
电子邮件	否	弹性	否
Web文档	可容忍	弹性	否
实时音视频	可容忍	音频: 5kbps-1Mbps 视频: 10kbps-5Mbps	是
可缓存音视频	可容忍	同上	是
交互游戏	可容忍	高于几kbps	是
即时消息	否	弹性	是或否

互联网传输层提供的服务

TCP服务:

- **面向连接:** 客户与服务器之间需要建立连接
- **可靠传输:** 可保证传递数据无差错
- **流量控制:** 发送数据不会超过接收端的容纳容量
- **拥塞控制:** 提供拥塞解决方案
- **不能提供:** 时延和带宽保证

UDP服务:

- **不可靠:** 不可靠的数据投递
- **不能提供:** 连接建立、可靠性、流量控制、拥塞控制、时延和带宽保证

Q: 为什么需要UDP?

互联网应用：常用应用使用的传输层服务

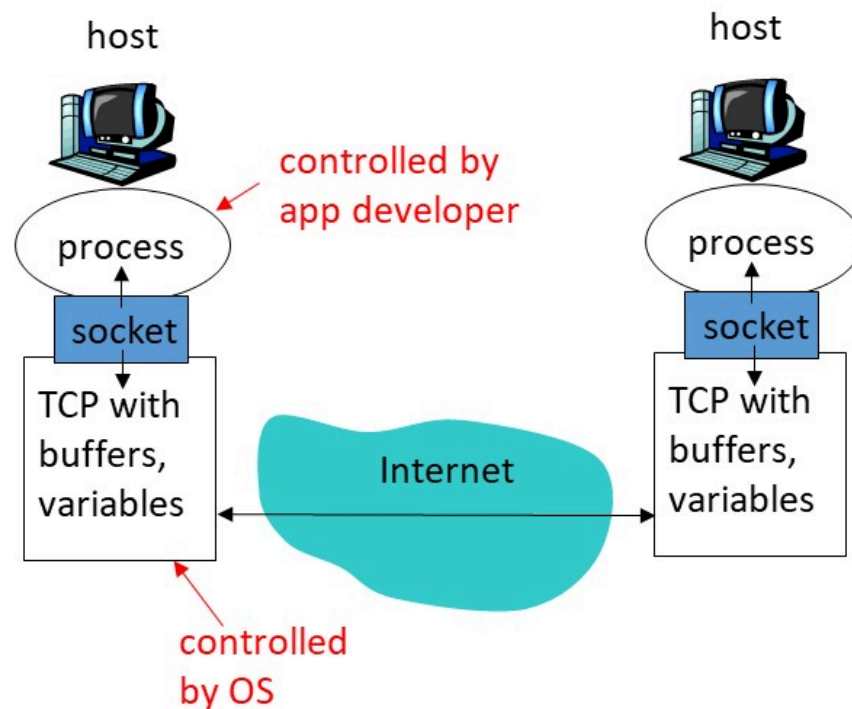
应用	应用层协议	传输层协议
电子邮件	SMTP [RFC 2821]	TCP
Web服务	HTTP [RFC 2616]	TCP
文件传输	FTP [RFC 959]	TCP
流媒体	HTTP、RTP	TCP or UDP
网络电话	RTP [RFC 1889] SIP、专有协议	典型为UDP

网络编程界面

- TCP/IP协议通常在操作系统的**内核**中实现
- **编程界面**：由操作系统提供的功能调用，可以使应用程序方便地使用内核的功能
- **socket（套接字）**：支持TCP/IP的操作系统为网络程序开发提供的**典型**网络编程界面

套接字sockets

- 进程通过**套接字**发送消息和接收消息
- 套接字可以看成一道“门”
 - 发送进程把消息从“门”推出去
 - 发送进程推出去的消息利用下层的通信设施传递到接收进程所在的“门”
 - 接收进程再从“门”把消息拉进去
- **API:** (1) 选择使用的传输层协议; (2) 对套接字的一些参数进行修改



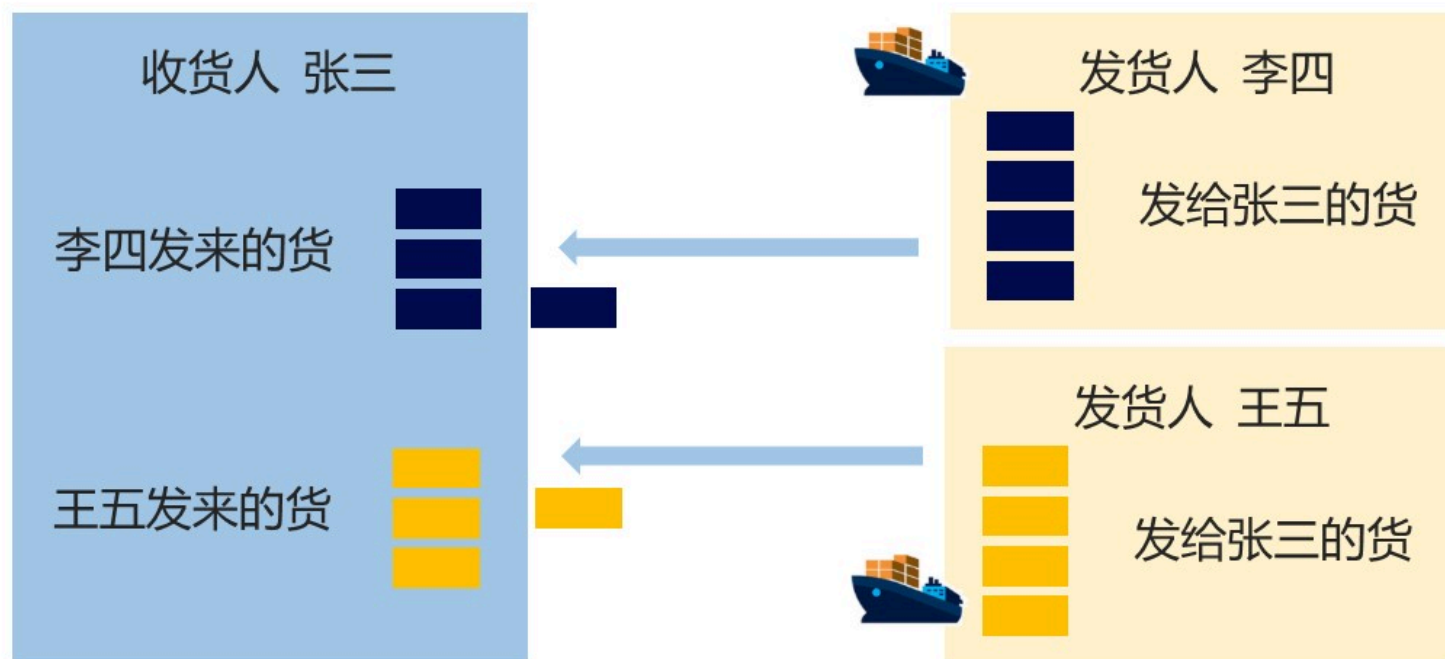
套接字sockets

- **数据报套接字**（datagram sockets）：使用UDP协议，支持主机之间面向非连接、不可靠的数据传输
- **流式套接字**（stream sockets）：使用TCP协议，支持主机之间面向连接的、顺序的、可靠的、全双工字节流传输
- 支持socket的**操作系统**：Windows、UNIX、Linux、iOS、Android等
- 支持socket的**编程语言**：C、C++、Python、Java、VB等

提前了解：TCP和UDP的区别



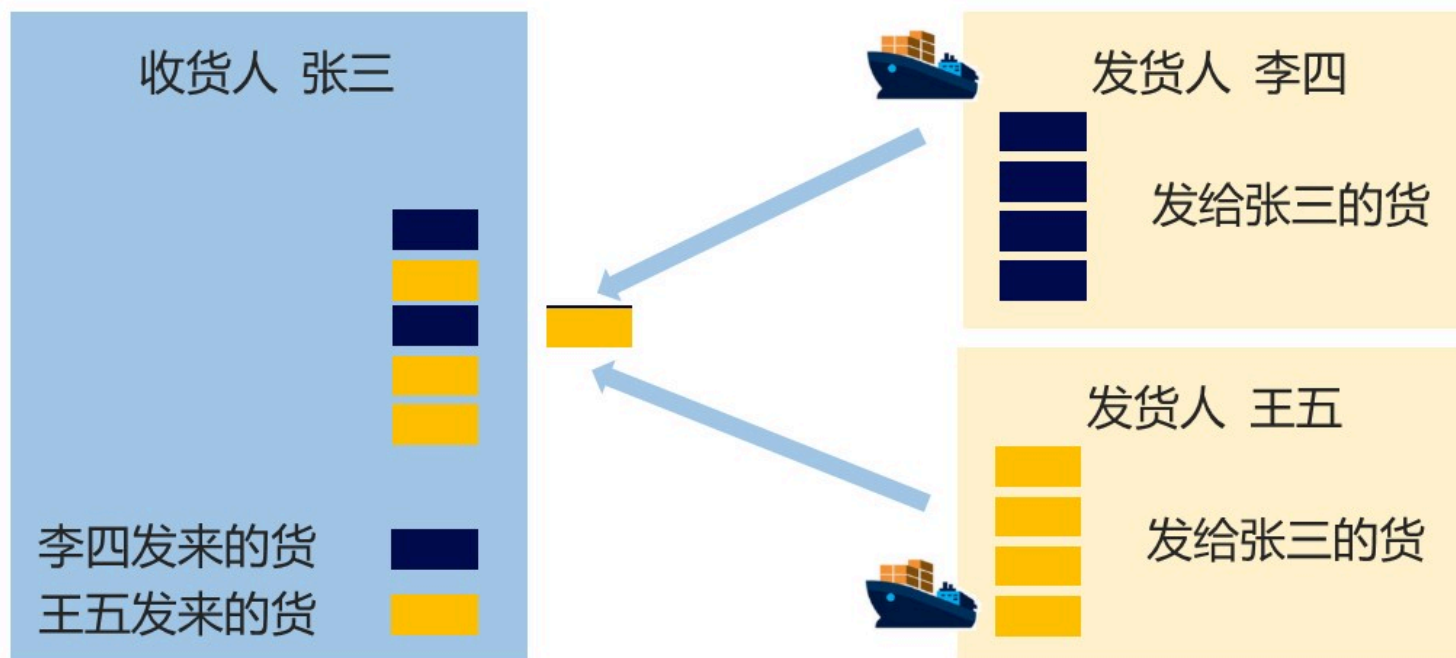
TCP传输示意图



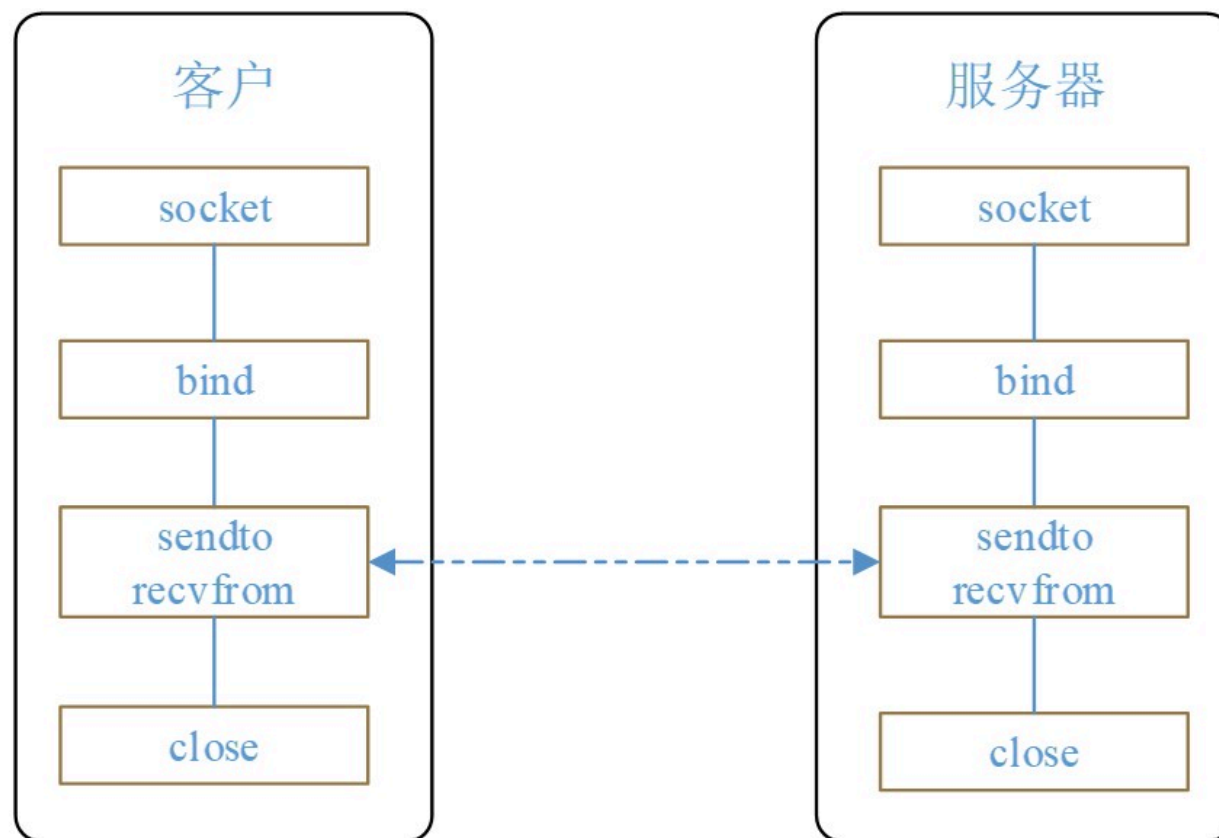
提前了解：TCP和UDP提供服务的区别



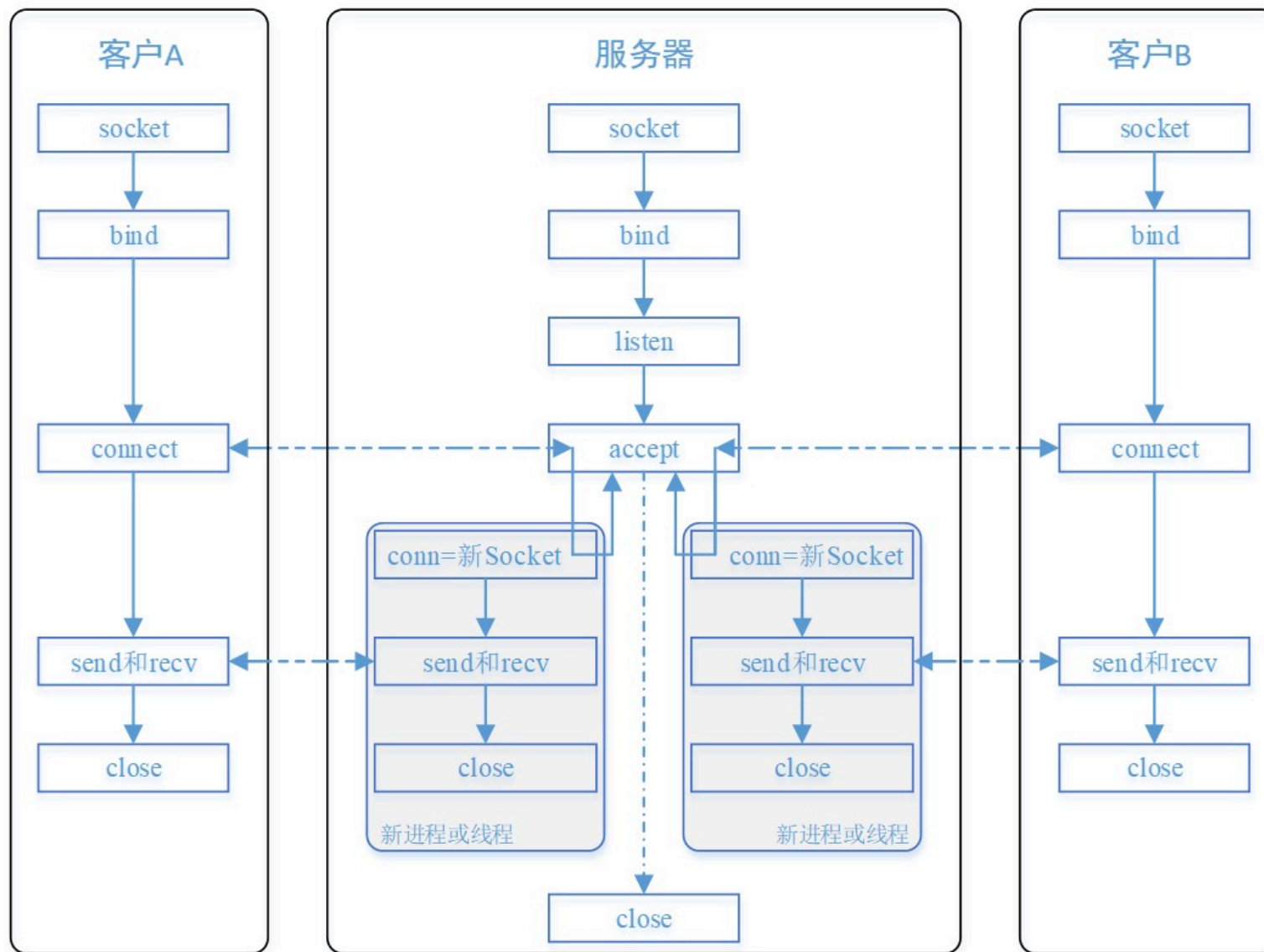
UDP传输示意图



2.3 利用UDP服务的应用程序编写步骤



2.3 利用TCP服务的应用程序编写步骤



WSAStartup

```
int WSAAPI WSAStartup (                //成功返回0， 否则为错误代码
    WORD wVersionRequested,            //调用者希望使用的最高版本
    LPWSADATA lpWSADATA                //可用Socket的详细信息
);
```

■ **功能：** 初始化Socket DLL， 协商使用的Socket版本

■ **WSADATA:**

- wVersion: 推荐调用者使用的Socket版本号
- wHighVersion: 系统实现的Socket最高版本号

■ 如果调用成功， 不再使用时需要调用WSACleanup释放Socket DLL资源

2.3 Socket编程 – C中常用的Socket函数



WSAStartup

Caller version support	Winsock DLL version support	wVersion requested	wVersion returned	wHighVersion returned	End result
1.1	1.1	1.1	1.1	1.1	use 1.1
1.0 1.1	1.0	1.1	1.0	1.0	use 1.0
1.0	1.0 1.1	1.0	1.0	1.1	use 1.0
1.1	1.0 1.1	1.1	1.1	1.1	use 1.1
1.1	1.0	1.1	1.0	1.0	Application fails
1.0	1.1	1.0	—	—	WSAVERNOTSUPPORTED
1.0 1.1	1.0 1.1	1.1	1.1	1.1	use 1.1
1.1 2.0	1.0 1.1	2.0	1.1	1.1	use 1.1
2.0	1.0 1.1 2.0	2.0	2.0	2.0	use 2.0
2.0 2.2	1.0 1.1 2.0	2.2	2.0	2.0	use 2.0
2.2	1.0 1.1 2.0 2.1 2.2	2.2	2.2	2.2	use 2.2



WSACleanup

```
int WSAAPI WSAStartup(  
  
);
```

//成功返回0

- **功能：** 结束使用Socket，释放Socket DLL资源
- 调用失败后可利用WSAGetLastError获取详细错误信息

2.3 Socket编程 – C中常用的Socket函数



```
socket SOCKET WSAAPI socket(  
    int af,  
    int type,  
    int protocol  
);
```

- **功能：** 创建一个Socket，并绑定到一个特定的传输层服务
- **参数：**
 - af: 地址类型。AF_INET、AF_INET6等
 - type: 服务类型。SOCK_STREAM、SOCK_DGRAM等
 - Protocol: 协议。IPPROTO_TCP、IPPROTO_UDP、IPPROTO_ICMP等。
如为0，则由系统自动选择
- **返回：**
 - 正确: Socket描述符
 - 错误: INVALID_SOCKET。可通过WSAGetLastError获取错误详情

2.3 Socket编程 – C中常用的Socket函数



```
bind          int bind(  
              SOCKET          s,  
              const sockaddr  *addr,  
              int             namelen  
              );
```

■ **功能：** 将一个本地地址绑定到指定的Socket

■ **参数：**

- s: socket描述符。
- addr: 地址。包括IP地址和端口号。如为INADDR_ANY和in6addr_any, 则由系统自动分配。
- namelen: 地址长度。通常为sockaddr结构的长度

■ **返回：**

- 正确: 0
- 错误: SOCKET_ERROR。可通过WSAGetLastError获取错误详情

2.3 Socket编程 – C中常用的Socket函数



```
listen      int WSAAPI listen(  
            SOCKET      s,  
            int          backlog  
            );
```

■ **功能：** 使socket进入监听状态，监听远程连接是否到来

■ **参数：**

- s: socket描述符。
- backlog: 连接等待队列的最大长度

■ **返回：**

- 正确: 0
- 错误: SOCKET_ERROR。可通过WSAGetLastError获取错误详情

■ **使用：** 流方式，Server端

2.3 Socket编程 – C中常用的Socket函数



```
connect      int WSAAPI connect(  
              SOCKET          s,  
              const sockaddr   *name,  
              int              namelen  
            );
```

■ **功能：** 向一个特定的socket发出建连请求

■ **参数：**

- s: socket描述符。
- addr: 地址。包括IP地址和端口号。
- namelen: 地址长度。通常为sockaddr结构的长度

■ **返回：**

- 正确: 0
- 错误: SOCKET_ERROR。可通过WSAGetLastError获取错误详情

■ **使用：** 流方式、Client端

2.3 Socket编程 – C中常用的Socket函数



accept

```
SOCKET WINAPI accept(  
    SOCKET  
    sockaddr  
    int  
    s,  
    *addr,  
    *addrlen  
);
```

- **功能：** 接受一个特定socket请求等待队列中的连接请求
- **参数：**
 - s: socket描述符。
 - addr: 返回远程端地址。
 - namelen: 返回地址长度。
- **返回：**
 - 正确: 返回新连接的socket描述符。
 - 错误: INVALID_SOCKET。可通过WSAGetLastError获取错误详情
- **使用：** 流方式、Server端。通常运行后进入阻塞状态，直到连接请求到来

2.3 Socket编程 – C中常用的Socket函数



sendto

```
int WINAPI sendto(  
    SOCKET          s,  
    const char      *buf,  
    int             len,  
    int             flags,  
    const struct sockaddr *to,  
    int             tolen );
```

■ **功能：** 向特定的目的地发送数据

■ **参数：**

- s: socket描述符。 buf: 发送数据缓存区。
- len: 发送缓冲区的长度 flags: 对调用的处理方式, 如OOB等
- to: 目标socket的地址 tolen: 目标地址的长度

■ **返回：**

- 正确: 返回实际发送的字节数。
- 错误: SOCKET_ERROR。可通过WSAGetLastError获取错误详情

■ **使用：** 数据报方式。

2.3 Socket编程 – C中常用的Socket函数



```
recvfrom      int WSAAPI recvfrom(  
              SOCKET    s,  
              char      *buf,  
              int        len,  
              int        flags,  
              sockaddr   *from,  
              int        *fromlen );
```

■ **功能：** 从特定的目的地接收数据

■ **参数：**

- s: socket描述符。 buf: 接收数据的缓存区。
- len: 接收缓冲区的长度 flags: 对调用的处理方式, 如OOB等
- from: 源socket的地址 fromlen: 源地址的长度

■ **返回：**

- 正确: 接收到的字节数。
- 错误: SOCKET_ERROR。可通过WSAGetLastError获取错误详情

■ **使用：** 数据报方式。

2.3 Socket编程 – C中常用的Socket函数



send

```
int WINAPI send(  
    SOCKET      s,  
    const char *buf,  
    int         len,  
    int         flags );
```

■ **功能：** 向远程socket发送数据

■ **参数：**

- s: socket描述符。 buf: 发送数据缓存区。
- len: 发送缓冲区的长度 flags: 对调用的处理方式, 如OOB等

■ **返回：**

- 正确: 返回实际发送的字节数。
- 错误: SOCKET_ERROR。可通过WSAGetLastError获取错误详情

■ **使用：** 流方式。

2.3 Socket编程 – C中常用的Socket函数



closesocket `int WSAAPI closesocket(
 SOCKET s
);`

■ **功能：** 关闭一个存在的socket

■ **参数：**

➤ s: socket描述符。

■ **返回：**

➤ 正确：0。

➤ 错误：SOCKET_ERROR。可通过WSAGetLastError获取错误详情

SOCKADDR 结构

```
typedef struct sockaddr {  
    u_short sa_family;  
    char     sa_data[14];  
} SOCKADDR, *PSOCKADDR, *LPSOCKADDR;
```

SOCKADDR_IN 结构

```
typedef struct sockaddr_in {  
    short      sin_family;  
    u_short    sin_port;  
    struct in_addr sin_addr;  
    char       sin_zero[8];  
} SOCKADDR_IN, *PSOCKADDR_IN, *LPSOCKADDR_IN;
```

in_addr 结构

```
struct in_addr {  
    union {  
        struct {  
            u_char s_b1;  
            u_char s_b2;  
            u_char s_b3;  
            u_char s_b4;  
        } S_un_b;  
        struct {  
            u_short s_w1;  
            u_short s_w2;  
        } S_un_w;  
        u_long S_addr;  
    } S_un;  
};
```

Big-Endian和Little-Endian

- **Q:** 计算机中，整数是如何存储的？
- **Little-Endian:** 低位字节排放在内存的低地址端，高位字节排放在内存的高地址端。
- **Big-Endian:** 高位字节排放在内存的低地址端，低位字节排放在内存的高地址端。
- 例如：0x12 34 56 78在内存中的存放形式为：
 - 内存 低地址 -----> 高地址
 - Big-Endian: 0x12 | 0x34 | 0x56 | 0x78
 - Little-Endian: 0x78 | 0x56 | 0x34 | 0x12

Big-Endian和Little-Endian

■ 常见CPU的字节序

- Big-Endian : PowerPC、IBM、Sun
- Little-Endian : **x86**、DEC
- ARM既可工作在Big-Endian, 也可工作在Little-endian

■ 网络使用的字节序: 网络通信协议都使用*Big-Endian*编码序

■ 主机序与网络序的转换

- htons: host to net -- short
- htonl: host to net -- long
- ntohs: net to host -- short
- ntohl: net to host -- long

2.3 Socket编程 – 编程中应注意的问题



CreateThread

```
HANDLE CreateThread(  
    LPSECURITY_ATTRIBUTES    lpThreadAttributes,  
    SIZE_T                   dwStackSize,  
    LPTHREAD_START_ROUTINE    lpStartAddress,  
    __drv_aliasesMem LPVOID   lpParameter,  
    DWORD                     dwCreationFlags,  
    LPDWORD                   lpThreadId );
```

■ 功能：创建线程

■ 参数：

- lpThreadAttributes：返回句柄能否被继承。NULL为不能继承。
- dwStackSize：堆栈的初始大小。0为缺省大小。
- lpStartAddress：新线程的开始执行地址。自己线程函数的开始地址。
- lpParameter：传递给线程的参数。
- dwCreationFlags：控制线程的标志。0为立即执行。
- lpThreadId：指向进程标识符的指针。NULL为不返回该指针。

■ 返回：

- 正确：新创建线程的句柄。
- 错误：NULL。可通过GetLastError获取错误详情

2.3 Socket编程 – 举例（数据报客户端）



```
void main()
{
    WSStartup(wVersionRequested, &wsaData);

    SOCKET sockClient = socket(AF_INET, SOCK_DGRAM, 0);

    SOCKADDR_IN addrSrv;

    sendto(sockClient, sendBuf, sendlen, 0, (SOCKADDR *)&addrSrv, len);

    recvfrom(sockClient, recvBuf, 50, 0, (SOCKADDR *)&addrSrv, &len);

    closesocket(sockClient);
    WSACleanup();
}
```

2.3 Socket编程 – 举例（数据报服务端）



```
void main()
{
    WSStartup(wVersionRequested, &wsaData);

    SOCKET sockSrv = socket(AF_INET, SOCK_DGRAM, 0);

    SOCKADDR_IN addrSrv;
    bind(sockSrv, (SOCKADDR*)&addrSrv, sizeof(SOCKADDR));

    SOCKADDR_IN addrClient; //远程IP地址
    loop {
        recvfrom(sockSrv, recvBuf, 50, 0, (SOCKADDR *)&addrClient, &len);
        sendto(sockSrv, sendBuf, sendlen, 0, (SOCKADDR *) &addrClient, len);
    }

    closesocket(sockClient);
    WSACleanup();
}
```


2.3 Socket编程 – 举例（流式客户端）



```
void main()
{
    WSStartup(wVersionRequested, &wsaData);

    SOCKET sockClient = socket(AF_INET, SOCK_STREAM, 0);

    connect(sockClient, (SOCKADDR*)&addrSrv, sizeof(SOCKADDR));

    recv(sockClient, recvBuf, 50, 0);
    send(sockClient, "hello", sendlen, 0);

    closesocket(sockClient);
    WSACleanup();
}
```


2.3 Socket编程 – 举例（流式服务端）



```
void main()
{
    WSStartup(wVersionRequested, &wsaData);

    SOCKET sockSrv = socket(AF_INET, SOCK_STREAM, 0);

    bind(sockSrv, (SOCKADDR*)&addrSrv, sizeof(SOCKADDR));

    listen(sockSrv, 5);

    while (1)
    {
        SOCKET sockConn = accept(sockSrv, (SOCKADDR*)&addrClient, &len);
        send(sockConn, sendBuf, strlen, 0);
        recv(sockConn, recvBuf, 50, 0);
        closesocket(sockConn);
    }

    closesocket(sockSrv);
    WSACleanup();
}
```

2.3 Socket编程 – 举例（多线程流式服务端）



```
void main()
{
    WSStartup(wVersionRequested, &wsaData);
    SOCKET sockSrv = socket(AF_INET, SOCK_STREAM, 0);
    bind(sockSrv, (SOCKADDR*)&addrSrv, sizeof(SOCKADDR));
    listen(sockSrv, 5);
    while (1) {
        SOCKET sockConn = accept(sockSrv, (SOCKADDR*)&addrClient, &len);
        hThread = CreateThread(NULL, NULL, handlerRequest, LPVOID(sockConn), 0, &dwThreadId);
        CloseHandle(hThread);
    }
    closesocket(sockSrv);
    WSACleanup();
}

DWORD WINAPI handlerRequest(LPVOID lparam)
{
    SOCKET ClientSocket = (SOCKET) (LPVOID) lparam;
    send(ClientSocket, sendBuf, strlen, 0);
    recv(ClientSocket, recvBuf, 50, 0);
    closesocket(ClientSocket);
    return 0;
}
```

■ **socketserver模块**：网络服务器编程架构

- TCPServer类：针对TCP服务器编程
- UDPServer类：针对UDP服务器编程

■ **TCPServer类与UDPServer类的使用**

- 编写请求处理类
- 创建TCPServer或UDPServer对象
- 利用TCPServer类或UDPServer类的server_forever()方法进行多次循环处理

2.3 Socket编程 – Python Socket编程举例

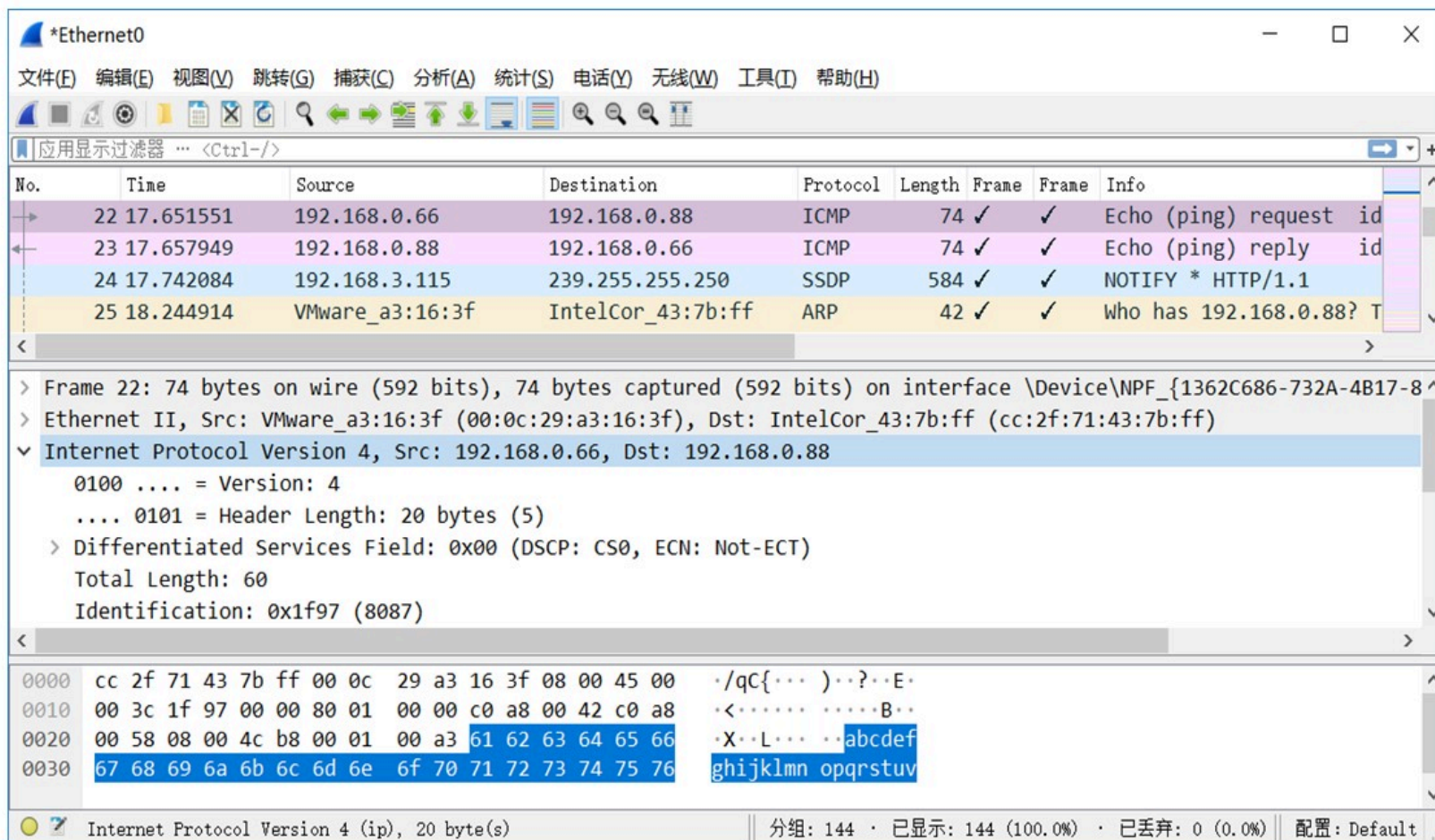


```
1      class MyTCPHandler(socketserver.BaseRequestHandler):
2          #重写基类中的handle函数
3          def handle(self):
4              #收发数据使用的socket存储在self.request中。
5              self.sock=self.request
6              .....
7              #接收数据。
8              self.recv_data=self.sock.recv(1024)
9              .....
10             #发送数据
11             self.sock.sendall(self.send_data)
12     .....
```

主程序

```
1      # 创建TCP服务器，为该服务器绑定的IP地址为host，端口号为port
2      with socketserver.TCPServer((host,int(port)), MyTCPHandler) as server:
3          # 循环进行收发处理，并在不用时自动关闭
4          server.serve_forever()
```


Wireshark



捕获数据包、设置显示过滤规则、设置捕获范围、查看统计信息

- **ping**: 测试网络连通性，通过发送ICMP回显请求与应答检测目标主机是否可达，并测量网络延迟、丢包率等
- **tracert**: 追踪数据包从本地计算机到目标地址所经过的路径，并记录每跳的延迟时间
- **ipconfig**: 查看IP地址、子网掩码、默认网关等基础网络信息
- **route**: 查看、添加、修改和删除路由表
- **arp**: 管理和查询主机的ARP缓存表
- **nslookup**: 查询域名系统记录，获取域名对应的IP地址或反向解析
- **netstat**: 显示网络连接、路由表及接口统计信息